BLOCKSEC

Security Audit Report for FeeEarner-smart-contract

Date: November 4, 2025 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction **1**
1.1 About Target Contracts . . . 1
1.2 Disclaimer . . . 1
1.3 Procedure of Auditing . . . 2
1.3.1 Security Issues . . . 2
1.3.2 Additional Recommendation . . . 2
1.4 Security Model . . . 3

Chapter 2 Findings **4**
2.1 Security Issue . . . 4
2.1.1 Token removal vulnerability due to malicious donations . . . 4
2.1.2 Lack of duplicate check in function `initialize()` . . . 6
2.1.3 `LiquidityManager` can withdraw funds when the contract is paused . . . 7
2.2 Recommendation . . . 8
2.2.1 Lack of validation for identical manager address update . . . 8
2.2.2 Lack of `isTokenAllowed` validation in the function `withdraw()` . . . 8
2.2.3 Apply the Checks-Effects-Interactions pattern . . . 9
2.3 Note . . . 9
2.3.1 Incompatibility with fee-on-transfer tokens . . . 9
2.3.2 Potential centralization risks . . . 10
2.3.3 Users are unable to withdraw contributed funds . . . 10

Report Manifest

Item	Description
Client	1Perp
Target	FeeEarner-smart-contract

Version History

Version	Date	Description
1.0	November 4, 2025	First release

Signature

Digitally signed by: BlockSec (https://blocksec.com)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-04-30 13:49:31+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at Email, Twitter and Medium.

Chapter 1 Introduction

1.1 About Target Contracts

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The target of this audit is the code repository [1] of FeeEarner-smart-contract of 1Perp.

The FeeEarner smart contract is an upgradeable, secure sponsorship pool. It enables users to contribute whitelisted ERC20 tokens (e.g., USDC/USDT), with all contributions tracked on-chain. The contract employs a robust role-based access control system featuring an owner for configuration and a Liquidity Manager for fund withdrawals. Architecturally, it utilizes the UUPS proxy pattern for upgrades, Pausable functionality, and ReentrancyGuard for enhanced security, ensuring a safe and efficient platform for managing community contributions.

Note this audit only focuses on the smart contracts in the following directories/files:

- contracts/FeeEarner.sol

Other files are not within the scope of the audit. Additionally, all dependencies of the smart contracts within the audit scope are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (`Version 1`), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (`Version 0`), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
FeeEarner-smart-contract	`Version 1`	`67bbf81080c1c7d9ff1baa8eb501004fd29cb715`
	`Version 2`	`707df03739360f5913d4a85dc83bee583034cb75`

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any war-

[1] https://github.com/1Perp/FeeEarner-smart-contract

ranties on discovering all security issues of the smart contracts, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of smart contracts.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan smart contracts with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of smart contracts and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

* Access control
* Permission management
* Whitelist and blacklist mechanisms
* Initialization consistency
* Improper use of the proxy system
* Reentrancy
* Denial of Service (DoS)
* Untrusted external call and control flow
* Exception handling
* Data handling and flow
* Events operation
* Error-prone randomness
* Oracle security
* Business logic correctness
* Semantic and functional consistency
* Emergency mechanism
* Economic and incentive impact

1.3.2 Additional Recommendation

* Gas optimization

* Code quality and style

Note *The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.*

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology [2] and Common Weakness Enumeration [3]. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	Likelihood: *High*	Likelihood: *Low*
High	High	Medium
Low	Medium	Low

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

[2] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[3] https://cwe.mitre.org/

Chapter 2 Findings

In total, we found **three** potential security issues. Besides, we have **three** recommendations and **three** notes.

- Low Risk: 3
- Recommendation: 3
- Note: 3

ID	Severity	Description	Category	Status
1	Low	Token removal vulnerability due to malicious donations	Security Issue	Fixed
2	Low	Lack of duplicate check in function `initialize()`	Security Issue	Fixed
3	Low	`LiquidityManager` can withdraw funds when the contract is paused	Security Issue	Fixed
4	-	Lack of validation for identical manager address update	Recommendation	Fixed
5	-	Lack of `isTokenAllowed` validation in the function `withdraw()`	Recommendation	Fixed
6	-	Apply the Checks-Effects-Interactions pattern	Recommendation	Fixed
7	-	Incompatibility with fee-on-transfer tokens	Note	-
8	-	Potential centralization risks	Note	-
9	-	Users are unable to withdraw contributed funds	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Token removal vulnerability due to malicious donations

Severity Low

Status Fixed in `Version 2`

Introduced by `Version 1`

Description The contract `FeeEarner` implements the function `removeAllowedToken()`, which requires zero token balance before removing a token from the whitelist. Meanwhile, the withdrawal functions are restricted to the liquidity manager role, which may not promptly remove donated tokens due to its potentially automated nature. In this case, the attacker can choose to donate 1 wei after `withdrawAll`/`withdraw` operations were executed to prevent token removal.

```
function removeAllowedToken(address token) external onlyOwner {
    if (!isTokenAllowed[token]) revert TokenNotFound();

    // Check if contract has any balance of this token
    uint256 balance = IERC20(token).balanceOf(address(this));
```

```
    if (balance > 0) revert TokenHasBalance();

    isTokenAllowed[token] = false;

    // Remove from array
    for (uint256 i = 0; i < allowedTokens.length; i++) {
        if (allowedTokens[i] == token) {
            allowedTokens[i] = allowedTokens[allowedTokens.length - 1];
            allowedTokens.pop();
            break;
        }
    }

    emit TokenRemoved(token);
}
```

Listing 2.1: contracts/FeeEarner.sol

```
function withdraw(address token, uint256 amount)
    external
    onlyLiquidityManager
    nonReentrant
{
    if (token == address(0)) revert ZeroAddress();
    if (amount == 0) revert ZeroAmount();

    uint256 balance = IERC20(token).balanceOf(address(this));
    if (balance < amount) revert InsufficientBalance();

    IERC20(token).safeTransfer(liquidityManager, amount);

    emit Withdrawal(token, amount, liquidityManager);
}

/**
 * @notice Withdraw all tokens from the contract
 */
function withdrawAll() external onlyLiquidityManager nonReentrant {
    uint256 length = allowedTokens.length;

    for (uint256 i = 0; i < length; i++) {
        address token = allowedTokens[i];
        uint256 balance = IERC20(token).balanceOf(address(this));

        if (balance > 0) {
            IERC20(token).safeTransfer(liquidityManager, balance);
            emit Withdrawal(token, balance, liquidityManager);
        }
    }
}
```

Listing 2.2: contracts/FeeEarner.sol

Impact Attackers can sabotage the token management system by donating dust amounts that effectively prevent tokens from being removed.

Suggestion Revise the logic accordingly.

Feedback from the project The project team states that the roles `Owner` and the `Liquidity Manager` are intended to be held by the same individual or entity, or by two team members who can communicate frequently to ensure proper coordination. Prior confirmation will be conducted before invoking the function `withdrawAndRemoveToken()`. The current permission configuration for the function `withdrawAndRemoveToken()` will ensure the compatibility with the functions `removeAllowedToken()` and `withdraw()`.

2.1.2 Lack of duplicate check in function `initialize()`

Severity Low

Status Fixed in `Version 2`

Introduced by `Version 1`

Description Function `initialize()`processes an array of token addresses through iterative calls to the function `_addToken()`. However, neither function contains checks to prevent duplicate token addresses from being added to the `allowedTokens` array. Once duplicates are added, they cannot be fully removed through the function `removeAllowedToken()` since this function can only delete one instance per call and sets `isTokenAllowed[token]` to false, preventing removal of remaining duplicates.

```
function initialize(
    address initialOwner,
    address initialLiquidityManager,
    address[] memory initialTokens
) public initializer {
    if (initialOwner == address(0)) revert ZeroAddress();
    if (initialLiquidityManager == address(0)) revert ZeroAddress();
    if (initialTokens.length > MAX_TOKENS) revert MaxTokensReached();

    __Ownable_init(initialOwner);
    __Ownable2Step_init();
    __Pausable_init();
    __ReentrancyGuard_init();
    __UUPSUpgradeable_init();

    liquidityManager = initialLiquidityManager;

    // Add initial tokens
    for (uint256 i = 0; i < initialTokens.length; i++) {
        _addToken(initialTokens[i]);
    }
}
```

Listing 2.3: contracts/FeeEarner.sol

```
function _addToken(address token) private {
```

```
    if (token == address(0)) revert ZeroAddress();

    allowedTokens.push(token);
    isTokenAllowed[token] = true;

    emit TokenAdded(token);
}
```

Listing 2.4: contracts/FeeEarner.sol

Impact Duplicate entries can cause operational inefficiencies and complicate token management throughout the contract's lifecycle.

Suggestion Add a check to prevent duplication in function `initialize()`.

2.1.3 `LiquidityManager` can withdraw funds when the contract is paused

Severity Low

Status Fixed in `Version 2`

Introduced by `Version 1`

Description Functions `withdraw()` and `withdrawAll()` enable token transfers to the liquidity manager. However, these functions lack the `whenNotPaused` modifier, creating a security contradiction where fund withdrawals remain operational during emergency pauses.

```
function withdraw(address token, uint256 amount)
    external
    onlyLiquidityManager
    nonReentrant
{
    if (token == address(0)) revert ZeroAddress();
    if (amount == 0) revert ZeroAmount();

    uint256 balance = IERC20(token).balanceOf(address(this));
    if (balance < amount) revert InsufficientBalance();

    IERC20(token).safeTransfer(liquidityManager, amount);

    emit Withdrawal(token, amount, liquidityManager);
}
```

Listing 2.5: contracts/FeeEarner.sol

```
function withdrawAll() external onlyLiquidityManager nonReentrant {
    uint256 length = allowedTokens.length;

    for (uint256 i = 0; i < length; i++) {
        address token = allowedTokens[i];
        uint256 balance = IERC20(token).balanceOf(address(this));

        if (balance > 0) {
            IERC20(token).safeTransfer(liquidityManager, balance);
            emit Withdrawal(token, balance, liquidityManager);
```

```
        }
    }
}
```

Listing 2.6: contracts/FeeEarner.sol

Impact If the `liquidityManager` is maliciously controlled, it can still withdraw funds from the contract even when it is paused. This limitation reduces the effectiveness of the emergency pause mechanism for protecting protocol assets.

Suggestion Apply the `whenNotPaused` modifier to both the functions `withdraw()` and `withdrawAll()`.

2.2 Recommendation

2.2.1 Lack of validation for identical manager address update

Status Fixed in `Version 2`

Introduced by `Version 1`

Description Function `setLiquidityManager()` allows the owner to update the liquidity manager address. However, the function does not validate whether the `newManager` parameter differs from the current `liquidityManager` address, creating a logical redundancy where the same address can be set repeatedly.

```
function setLiquidityManager(address newManager) external onlyOwner {
    if (newManager == address(0)) revert ZeroAddress();

    address oldManager = liquidityManager;
    liquidityManager = newManager;

    emit LiquidityManagerUpdated(oldManager, newManager);
}
```

Listing 2.7: contracts/FeeEarner.sol

Suggestion Add a check to prevent the same address to be set repeatedly.

2.2.2 Lack of `isTokenAllowed` validation in the function `withdraw()`

Status Fixed in `Version 2`

Introduced by `Version 1`

Description Function `withdraw()` enables the liquidity manager to transfer tokens from the contract. However, the function does not verify whether the `isTokenAllowed[token]` is true, creating a security gap where any ERC20 token held by the contract can be withdrawn.

```
function withdraw(address token, uint256 amount)
    external
    onlyLiquidityManager
    nonReentrant
{
```

```
    if (token == address(0)) revert ZeroAddress();
    if (amount == 0) revert ZeroAmount();

    uint256 balance = IERC20(token).balanceOf(address(this));
    if (balance < amount) revert InsufficientBalance();

    IERC20(token).safeTransfer(liquidityManager, amount);

    emit Withdrawal(token, amount, liquidityManager);
}
```

Listing 2.8: contracts/FeeEarner.sol

Suggestion Add a check to verify whether the `isTokenAllowed[token]` is true.

2.2.3 Apply the Checks-Effects-Interactions pattern

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract, the function `contribute()` performs an external call `safeTransferFrom()` before updating the internal state variable `userContributions`, which violates the Checks-Effects-Interactions pattern.

```
function contribute(address token, uint256 amount)
    external
    whenNotPaused
    nonReentrant
{
    if (token == address(0)) revert ZeroAddress();
    if (amount == 0) revert ZeroAmount();
    if (!isTokenAllowed[token]) revert TokenNotAllowed();

    // Transfer tokens from user to contract
    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);

    // Update contributions
    userContributions[msg.sender][token] += amount;
    totalContributions[token] += amount;

    emit Contribution(msg.sender, token, amount);
}
```

Listing 2.9: contracts/FeeEarner.sol

Suggestion Follow the CEI pattern to ensure that internal state updates occur before any external interactions, preventing potential reentrancy related risks.

2.3 Note

2.3.1 Incompatibility with fee-on-transfer tokens

Introduced by `Version 1`

Description Function `contribute()` records the full amount parameter in its accounting mappings (`userContributions` and `totalContributions`) after a token transfer. Note, this accounting method is incompatible with fee-on-transfer tokens, where the actual net amount received by the contract is less than the specified amount due to protocol fees deducted during transfer.

```
function contribute(address token, uint256 amount)
    external
    whenNotPaused
    nonReentrant
{
    if (token == address(0)) revert ZeroAddress();
    if (amount == 0) revert ZeroAmount();
    if (!isTokenAllowed[token]) revert TokenNotAllowed();

    // Transfer tokens from user to contract
    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);

    // Update contributions
    userContributions[msg.sender][token] += amount;
    totalContributions[token] += amount;

    emit Contribution(msg.sender, token, amount);
}
```

Listing 2.10: contracts/FeeEarner.sol

2.3.2 Potential centralization risks

Introduced by `Version 1`

Description In this project, several privileged roles (e.g., `liquidityManager`) can conduct sensitive operations, which introduces potential centralization risks. For example, The `liquidityManager` can withdraw all tokens from the contract at any time using the functions `withdraw()` and `withdrawAll()` based on the protocol. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

2.3.3 Users are unable to withdraw contributed funds

Introduced by `Version 1`

Description Tokens that users deposit into the contract `FeeEarner` through the function `contribute()` currently cannot be directly withdrawn by users through contract functionality.

BOOST WEB3 THROUGH NEXT-GENERATION SECURITY & USABILITY INNOVATIONS